Classes, Iterators and Generators

Comprehensions, objects that keep state between calls, the methods Python calls behind ordinary syntax, the iterator protocol, and functions that produce values one at a time. Standard library only.

01 Comprehensions
02 State between calls
03 Writing a class
04 Dunder methods
05 The iterator protocol
06 Generators

Scope. Two constructions: an object that keeps state between calls, which a plain function does not, and a function that produces values one at a time, where a list holds every item at once. Both are already used from the outside, in a `for` loop over a file, in `len` on a list and in a `range` that does not fill memory.

After reading it you should be able to do the following without looking anything up.

- Write a class with `__init__` and methods that change its own state.
- Say what `len(x)`, `x in y` and `y[i]` call, and implement `__len__`, `__contains__` and `__getitem__` so that the built-ins work on instances of a user-defined class.
- Implement the iterator protocol by hand: `__iter__`, `__next__`, and raising `StopIteration`.
- Write a generator with `yield`, and say why it does not build a list.
- Say when a generator is the better choice and when a list is.

01

Comprehensions

An expression that builds a container by looping, shorter than the loop it replaces.

COMPREHENSION FORMS

```
nums  = [4, -1, 7, -9, 0, -3, 5]
words = ["alpha", "beta", "gamma"]

squares  = [x * x for x in nums]          # list comprehension   -> a list of 7
distinct = {x * x for x in nums}          # set comprehension    -> a set of 7
sized    = {w: len(w) for w in words}     # dict comprehension   -> 3 key: value pairs
lazy     = (x * x for x in nums)          # generator expression -> a generator, no items yet
```

The first three build the whole container immediately. The fourth computes nothing until items are requested from it.

A comprehension is always equivalent to a loop. These two are the same program:

```
# nums: a list of 7 ints, 3 of them negative

kept = [x for x in nums if x >= 0]   # -> [4, 7, 0, 5]

kept = []
for x in nums:
    if x >= 0:
        kept.append(x)               # -> [4, 7, 0, 5]
```

The optional `if` at the end filters, so the result can be shorter than the input. It is also the efficient way to remove elements from a list: building a new list is one pass, while deleting from the middle of an existing one shifts every later element down and costs a pass of its own each time.

LIMITS OF COMPREHENSIONS

A comprehension holds exactly one expression. When the body needs a second statement, a temporary variable or a `try`, the loop is the form to use. The loop is also the clearer form in two more cases:

- **Nesting past two levels.** A comprehension with three `for` clauses is harder to read than the loops it replaces.
- **Side effects.** A comprehension is for producing a value. `[print(x) for x in nums]` prints, and also builds a list of `None` values that is never used.

NESTED COMPREHENSIONS

The `for` clauses appear in the same order as the nested loops they stand for, but the output expression comes first.

```
grid = [[1, 2], [3, 4], [5, 6]]      # 3 rows of 2

flat = [x for row in grid for x in row]   # -> [1, 2, 3, 4, 5, 6]

flat = []
for row in grid:              # first clause is the outer loop
    for x in row:             # second clause is the inner loop
        flat.append(x)        # -> [1, 2, 3, 4, 5, 6]
```

02

State between calls

A result that depends on earlier calls needs data that persists from one call to the next.

A counter cannot be written as a plain function, because its local variables are created afresh on every call:

```
def count():
    total = 0        # created fresh on every call
    total += 1
    return total

count(), count(), count()      # -> (1, 1, 1)
```

The usual first repair is a global variable. There is only one of it, so two independent counters are impossible, and any part of the program can change it between calls.

```
total = 0

def count():
    global total
    total += 1
    return total
```

A class solves both problems. The data lives inside an object, each object has its own, and the methods that change it are defined in the same class:

```
class Counter:
    def __init__(self, start=0):
        self.total = start

    def count(self):
        self.total += 1
        return self.total

a = Counter()                       # a.total is 0
b = Counter(100)                    # b.total is 100
a.count(), a.count(), b.count()     # -> (1, 2, 101)
```

`a` and `b` are separate objects with separate state. A class is the form to use when there are several of something and each has to keep its own state.

STATELESS FUNCTIONS

If the output depends only on the arguments, a function is enough. A class whose `__init__` stores nothing and whose methods never read `self` adds nothing that plain functions do not have. State is needed only if two calls with the same arguments must be able to give different answers.

03

Writing a class

A class is a template. An instance is one object built from it. Everything an instance stores is held in attributes reached through `self`.

__INIT__ AND SELF

`__init__` runs immediately after a new instance is created, and sets up that instance's starting data. It is not quite a constructor: the object already exists by the time it runs, and it returns nothing.

`self` is the instance the method was called on. Python passes it automatically as the first argument, which is why it appears in the definition but not at the call site:

```
c = Counter(10)      # calls Counter.__init__(c, 10)
c.count()            # Counter.count(c)   -> 11
```

Attributes are created by assigning to them. There is no separate declaration, and an attribute can be added at any later point, though adding one outside `__init__` makes an object whose set of attributes depends on which methods have run.

```
class Playlist:
    def __init__(self, name):
        self.name = name            # these two lines create the attributes
        self.tracks = []

    def add(self, title):
        self.tracks.append(title)
        return self                 # returning self allows p.add("a").add("b")

    def total(self):
        return len(self.tracks)
```

The name `self` is a convention, not a keyword. Any name works in that position, and no other name is used in practice.

INSTANCE ATTRIBUTES AGAINST CLASS ATTRIBUTES

A name assigned inside `__init__` through `self` belongs to that one instance. A name assigned directly in the class body belongs to the class and is shared by every instance of it.

```
class Track:
    medium = "digital"              # class attribute: one copy, shared
    def __init__(self, title):
        self.title = title          # instance attribute: one copy per track
```

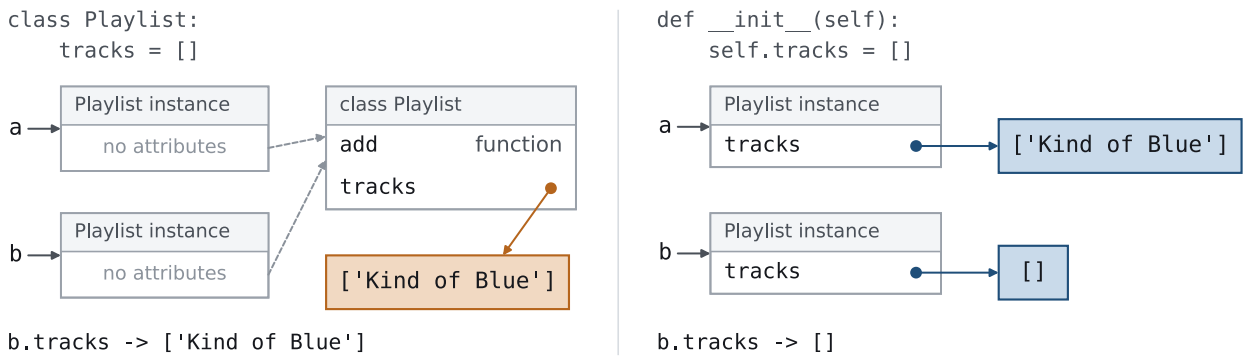
Sharing is harmless for a constant. It is a defect when the shared object can be modified, because every instance then sees the change, and it produces no error:

```
class Playlist:
    tracks = []                     # one list, shared by every Playlist instance

    def add(self, title):
        self.tracks.append(title)

a, b = Playlist(), Playlist()
a.add("Kind of Blue")
b.tracks                            # -> ['Kind of Blue'], although b was not touched
```

The fix is to create the list per instance, inside `__init__`.



Left: a and b have no attributes of their own, so a.tracks and b.tracks both resolve to the one list on the class. Right: with the list created in __init__, each instance holds its own.

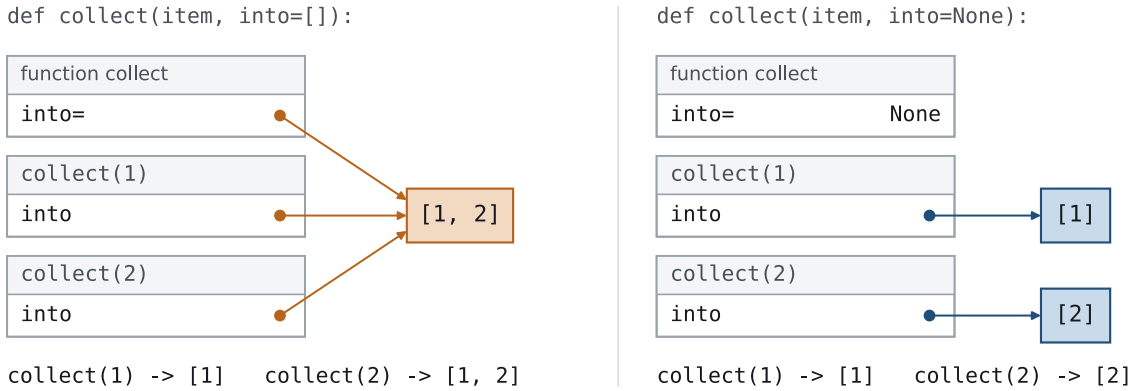
MUTABLE DEFAULT ARGUMENTS

A default argument is evaluated once, when the `def` is executed, not once per call. A mutable default is therefore shared between calls in exactly the way a mutable class attribute is shared between instances.

```
def collect(item, into=[]):         # this list is created once, when def runs
    into.append(item)
    return into

collect(1)                          # -> [1]
collect(2)                          # -> [1, 2], not [2]

def collect(item, into=None):       # the standard repair
    if into is None:
        into = []
    into.append(item)
    return into
```



The default list is created once, when def runs, and kept on the function; both calls that omit into append to it. With into=None, each call creates a new list.

__REPR__

By default, printing an object gives its type and memory address. `__repr__` replaces that with a string the class defines, and the same string appears for the object in a failing test's output.

```
class Playlist:
    def __init__(self, name):
        self.name = name
        self.tracks = []

    def __repr__(self):
        return f"Playlist({self.name!r}, {len(self.tracks)} tracks)"

Playlist("Roadtrip")                # -> Playlist('Roadtrip', 0 tracks)
```

In the `return` line, the `f` prefix marks an f-string, in which an expression inside braces is evaluated and inserted. The `!r` selects the `repr` of a value rather than its `str`, which puts the quotation marks around the name.

04

Dunder methods

Methods named with double underscores at both ends are called dunder methods. Python calls them when the matching syntax or built-in function is used.

SYNTAX	PYTHON CALLS	PURPOSE
`len(x)`	`x.__len__()`	The number of items
`v in x`	`x.__contains__(v)`	Membership
`x[i]`	`x.__getitem__(i)`	Indexing
`x[i] = v`	`x.__setitem__(i, v)`	Index assignment
`x == y`	`x.__eq__(y)`	Equality
`repr(x)`	`x.__repr__()`	The text shown in a console
`str(x)`, `print(x)`	`x.__str__()`	Text for a human, falls back to `__repr__`
`for v in x`	`x.__iter__()`	Iteration

Three of them turn the playlist into something that behaves like a built-in container:

```
class Playlist:
    def __init__(self, name):
        self.name = name
        self.tracks = []

    def add(self, title):
        self.tracks.append(title)

    def __len__(self):
        return len(self.tracks)

    def __contains__(self, title):
        return title in self.tracks

    def __getitem__(self, i):
        return self.tracks[i]

p = Playlist("Roadtrip")
p.add("Kind of Blue")
p.add("So What")                # p.tracks now holds 2 titles

len(p)                  # -> 2,     through __len__
"So What" in p          # -> True,  through __contains__
p[0]                    # -> 'Kind of Blue', through __getitem__
p[-1]                   # -> 'So What', the list handles the negative index
```

`__getitem__` forwards to the list, so negative indices and slices keep working without another line of code.

> **PARTIAL PROTOCOLS**
>
> These methods are independent. A class with `__len__` but no `__getitem__` supports `len(x)` and raises `TypeError` on `x[0]`. Python checks for the specific method at the moment the syntax is used, so an object supports exactly the operations it has methods for and nothing has to be declared in advance.

`__EQ__` AND `__HASH__`

By default two objects are equal only if they are the same object. Defining `__eq__` replaces that with the comparison the method implements:

```
class Track:
    def __init__(self, title, artist):
        self.title = title
        self.artist = artist

    def __eq__(self, other):
        if not isinstance(other, Track):
            return NotImplemented
        return (self.title, self.artist) == (other.title, other.artist)
```

For an unrelated type, `__eq__` returns `NotImplemented` rather than `False`. Python then tries the comparison the other way round, and compares identity if that also returns `NotImplemented`.

Defining `__eq__` removes the default `__hash__`, so instances can no longer go in a set or be used as dictionary keys:

```
{Track("So What", "Miles Davis")}
# TypeError: cannot use 'Track' as a set element (unhashable type: 'Track')
```

The removal is deliberate: a hash table only works if equal objects hash the same and a key's hash never changes. A class that redefines equality and has to stay hashable needs its own `__hash__`, built from the same fields:

```
    def __hash__(self):                          # inside Track, beside __eq__
        return hash((self.title, self.artist))   # the same two fields
```

An object meant to be mutable is left unhashable, for the same reason that `list` is unhashable and `tuple` is not.

05

The iterator protocol

A `for` loop is built on a protocol any object can implement: `__iter__`, `__next__` and `StopIteration`.

An **iterable** is any object that can be looped over: a list, a string, a dictionary, an open file. An **iterator** is the object that steps through the items and stores the current position. Two built-in functions connect them.

- `iter(x)` returns a fresh iterator for the iterable `x`, by calling its `__iter__` method.
- `next(it)` returns the next item of the iterator, by calling its `__next__` method. When nothing remains it raises `StopIteration` rather than returning a value.

So this loop:

```
nums = [4, -1, 7, -9, 0, -3, 5]

for item in nums:
    print(item)
```

is shorthand for exactly this:

```
# nums: a list of 7 ints

it = iter(nums)
while True:
    try:
        item = next(it)
    except StopIteration:
        break
    print(item)
```

Any object providing those two methods can be looped over. Written by hand:

```
class CountUpTo:
    def __init__(self, limit):
        self.limit = limit
        self.current = 0

    def __iter__(self):              # return the iterator, here the object itself
        return self

    def __next__(self):              # produce one item, or signal the end
        if self.current >= self.limit:
            raise StopIteration
        self.current += 1
        return self.current

for n in CountUpTo(3):
    print(n)                         # -> 1, 2, 3
```

ITERABLES AND ITERATORS

An object with both methods is an iterator, and its `__iter__` returns `self`. An object with only `__iter__` is an iterable that returns a separate iterator each time. An iterable of that kind can be looped over repeatedly; an iterator is exhausted after one pass.

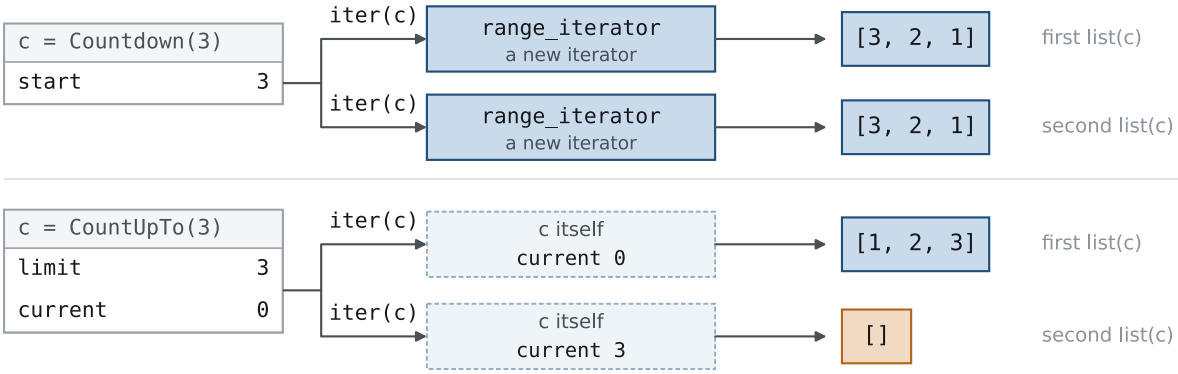
```
class Countdown:                     # an iterable: a fresh iterator per loop
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        return iter(range(self.start, 0, -1))

c = Countdown(3)
list(c)         # -> [3, 2, 1]
list(c)         # -> [3, 2, 1] again, because each loop got its own iterator

c = CountUpTo(3)                     # an iterator: one pass, then finished
list(c)         # -> [1, 2, 3]
list(c)         # -> [], because the state was consumed by the first pass
```

A list is an iterable and not an iterator: it has no `__next__`, and `iter(xs)` returns a separate list-iterator object every time it is called. That is why a list can be walked repeatedly and a generator cannot.



iter(c) on the iterable Countdown(3) returns a new range_iterator each time, so both passes produce [3, 2, 1]. On the iterator CountUpTo(3) it returns c itself, and the second pass starts at current 3 and produces [].

THE SEQUENCE PROTOCOL

If a class defines `__getitem__` that accepts 0, 1, 2 and raises `IndexError` when it runs out, Python will iterate it with no `__iter__` at all. The `Playlist` class, whose `__getitem__` forwards to its list of tracks, is iterable for this reason.

06

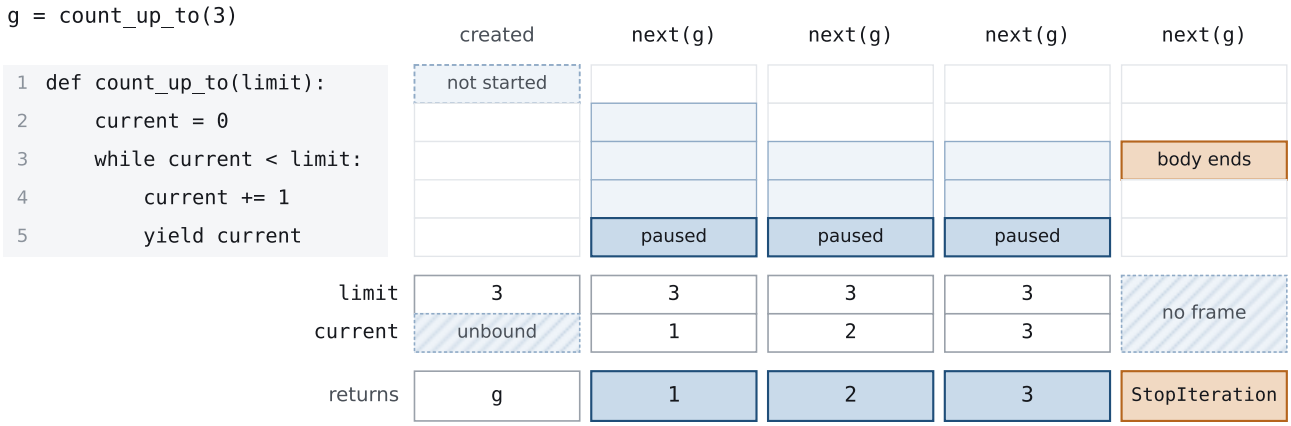
Generators

A function containing `yield` returns an iterator, the same kind of object that a class with `__iter__` and `__next__` builds by hand. The body runs only as items are requested.

A function containing `yield` is a generator function. Calling it does not run the body: it returns a generator object, which is an iterator. The body runs only as items are requested, pauses at each `yield`, and keeps its local variables alive until the next request.

```
def count_up_to(limit):
    current = 0
    while current < limit:
        current += 1
        yield current                # pause here, resume on the next request

for n in count_up_to(3):
    print(n)                         # -> 1, 2, 3, the same output as CountUpTo
```



Each next(g) runs the shaded lines of count_up_to(3) and pauses at the yield, with limit and current kept in the frame. The fourth call leaves the loop, the body ends and next(g) raises StopIteration.

The loop is unchanged. The generator supplies `__iter__` and `__next__`, and raises `StopIteration` when the function body ends, with none of that code written by hand.

THE PAUSED FRAME

The class version stored its position in `self.current`, by hand. The generator stores it in the paused frame: every local variable, and the exact line it stopped on.

Because only this frame is kept between items, a generator never holds more than the item it is producing at that moment, so it can stand for a sequence larger than memory, or one with no end:

```
def naturals():                      # infinite, one value per request
    n = 0
    while True:
        n += 1
        yield n

squares = (n * n for n in naturals())   # a generator, no items yet
[next(squares) for _ in range(5)]       # -> [1, 4, 9, 16, 25]
```

YIELD FROM

`yield from other_iterable` yields every item that iterable produces, one at a time, as if they had been yielded here directly.

```
def letters():
    yield "a"
    yield "b"

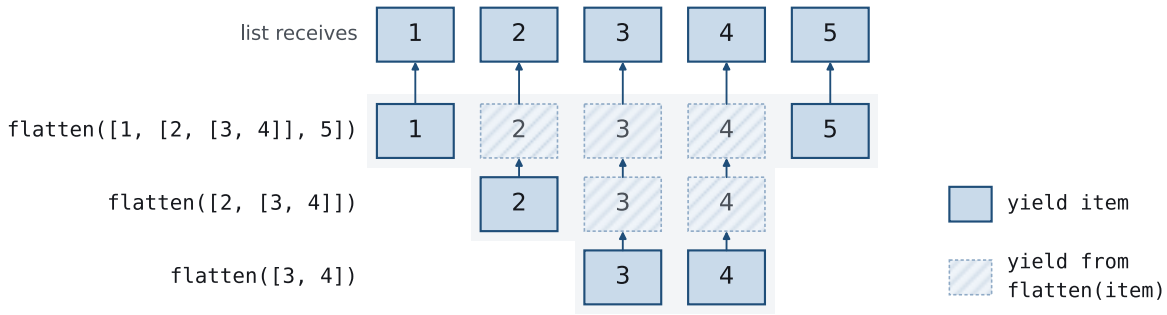
def both():
    yield from letters()             # delegate: a, then b
    yield from letters()             # and again

list(both())                         # -> ['a', 'b', 'a', 'b']
```

The common use is recursion: a generator that walks a structure nested to an unknown depth calls itself through `yield from`:

```
def flatten(items):
    for item in items:
        if isinstance(item, list):
            yield from flatten(item)
        else:
            yield item

list(flatten([1, [2, [3, 4]], 5]))   # -> [1, 2, 3, 4, 5]
```



One row per generator and one column per value of list(flatten([1, [2, [3, 4]], 5])). Each value is yielded by the generator for the list that holds it and passed up unchanged by each yield from above it; the grey bands mark when each generator exists.

RECURSING WITHOUT YIELD FROM

Calling a recursive generator without `yield from` or a loop creates a generator object and discards it. No exception is raised, the body of that inner call never runs, and every nested value is missing from the output.

```
def flatten(items):
    for item in items:
        if isinstance(item, list):
            flatten(item)            # builds a generator, discards it
        else:
            yield item

list(flatten([1, [2, [3, 4]], 5]))   # -> [1, 5], and no error
```

SINGLE-PASS LIMITATIONS

A generator is single pass. Once read, the sequence cannot be walked again, its length is unknown without consuming it, and there is no random access. A list supports all three, because it holds every item in memory.

```
g = (x * x for x in [1, 2, 3])   # a generator, no items yet
list(g)          # -> [1, 4, 9]
list(g)          # -> [], the first pass exhausted it
len(g)           # TypeError: object of type 'generator' has no len()
```

A generator suits a sequence that is read once from front to back; a sequence that is read repeatedly or out of order needs a list.

> #### RECURSION DEPTH
>
> Python limits recursion to 1000 frames by default and raises `RecursionError` beyond it. A recursive generator uses one frame per level of nesting, so it fails on a structure nested about a thousand levels deep. An explicit stack in a list has no such limit, since it lives on the heap.

LOADING DATA IN BATCHES

Training data often does not fit in memory, so it is loaded in batches: an object produces one batch at a time, on request, and never holds the whole set. A generator does this. PyTorch's `DataLoader` is an iterable rather than an iterator: each `for` loop over it gets a fresh iterator whose `__next__` returns the next batch. A file object is its own iterator, which is how a multi-gigabyte log file is read one line at a time rather than whole with `read()`.

FURTHER READING

- The classes tutorial, and its section on iterators and generators.
- The data model reference: the full list of dunder methods.
- itertools, a standard-library module of generators for the patterns that recur: chaining sequences, grouping runs, slicing, and taking the first few items of an infinite one.